A Wireless Data Streaming Device for a Temperature Programming Control System

Maxim LaPlante (CE, EE)

Purpose

The aim of this project is to provide a wireless data streaming tool for a piece of scientific instrumentation. The piece of instrumentation in question is a temperature programming and control system (TPCS) for isothermal, temperature-programmed desorption (TPD) experiments. The TPCS measures and controls crystal temperature as it increases at rates in excess of 100°K/s. Mass spectrometer and power supply output data complement temperature data. These are, at present, streamed over USB to a computer. They may, instead, be streamed serially to a wireless transmitter, which in turn streams data to a computer.

Overview

This is a system which allows a datalogging device to stream data wirelessly to a computer. It comprises two Raspberry Pi Pico 2 W boards which communicate using UDP over a WiFi connection. One board (the **transmitter**) provides a WiFi access point (AP) to which the other (the **receiver**) connects. Once a connection has been, the transmitter begins sending UDP packets with input data to the receiver, which in turn reports them serially to a computer through a USB serial port.

Operation

Data is streamed as raw bytes over the UDP protocol. Each datum is a 16-bit value, and as such requires two UDP bytes. The RP2350 microcontrollers are little endian, so the data is streamed in LSByte-first order. The receiver responds with a message of the same size for simplicity's sake. The response begins with an "ACK" string which is null-terminated. The transmitter uses this ACK message to keep track of the number of "lost" transmissions. It does not implement any recovery regime; lost messages are lost messages. Data is sent in batches. Batches are further divided into messages.

Software

The software for this system is built on two pieces of example software. These are the Raspberry Pi Pico SDK example for establishing an AP and the Hunter Adams Cornell University RP2040 demo for creating a UDP beacon. The former directly uses the lwip (lightweight internet protocol) stack, while the latter uses a PT (proto-threads) stack which sits atop the lwip stack. The transmitter establishes an AP and provides the receiver an IPv4 address on its own network. The receiver's given IP address is stored and used by the UDP threads in

addition to a UDP port (arbitrarily of value 1234). The transmitter itself uses the CYW (WiFi chip) default IP address, so the receiver uses that default as its "destination" IP address.

At present, data is sourced from the transmitter's onboard ADC. DMA channels are daisy-chained and double-buffered so that the data in each buffer may alternately be read in and read out. The ADC runs at a sample rate of 48kHz (i.e. with a clock divider of 1000 from the 48MHz ADC clock). The DMA buffers are each 256 samples deep and 2 bytes wide; they each comprise 512 bytes. Interrupt routines triggered by DMA channel completion modify global variables and manage signals to the PT scheduler. The scheduler maintains transmit and receive threads, but underlying UDP reads and writes are managed by callbacks and buffers under the hood, respectively.

System Performance Metrics

Once the system was brought to a point where ADC data seemed to be streaming, the first and most glaring issue was loss. That is to say, only a fraction of the messages being transmitted were received. This prompted an analysis of the tradeoffs having to do with UDP message length. Loss, throughput, and round-trip time were analyzed for message lengths of 512, 256, and 128 bytes. These message lengths correspond to 1, 2, and 4 messages per batch (i.e. DMA buffer).

Throughput

Theoretically, the throughput for this system ought to match the input data rate. This rate is 48ksps (the ADC sample rate) times 16 bits per sample (the size of the data[1]). This translates to a data rate of 96kBps or 768kbps. The actual throughput of the system is extrapolated by comparing the number of DMA "cycles" (i.e. DMA interrupts) to the number of transmission events divided by the ratio of message and DMA buffer sizes. The difference between these is the "DMA skip" figure and quantifies the ability of the transmitter to keep up with input data.

Loss

Loss was quantified similarly. UDP receive events were counted and compared to transmission events as recorded by the transmitter. Since the transmitter only receives ACK messages from the receiver, this is a good proxy for transmission success. The "ACK skip" figure therefore quantifies the loss rate of the system.

Round-Trip Time

Round-trip time was also quantified by comparing a transmit and receive timestamp generated by the transmitter. This difference theoretically captures round-trip time, but the asynchronous mechanisms underlying the software complicated this somewhat. Still, a general

[1] The ADC itself has a resolution of 12 bits.

idea of the differences in round-trip times (including some of the task of processing data on the part of the receiver) due to differences in message length can be ascertained.

Results

ACK and DMA skips were reported every 100 transmission events. For a message size of 127, where four messages are sent per batch, there was significant ACK skippage. DMA skippage was low and remained fixed across time for all cases tested.

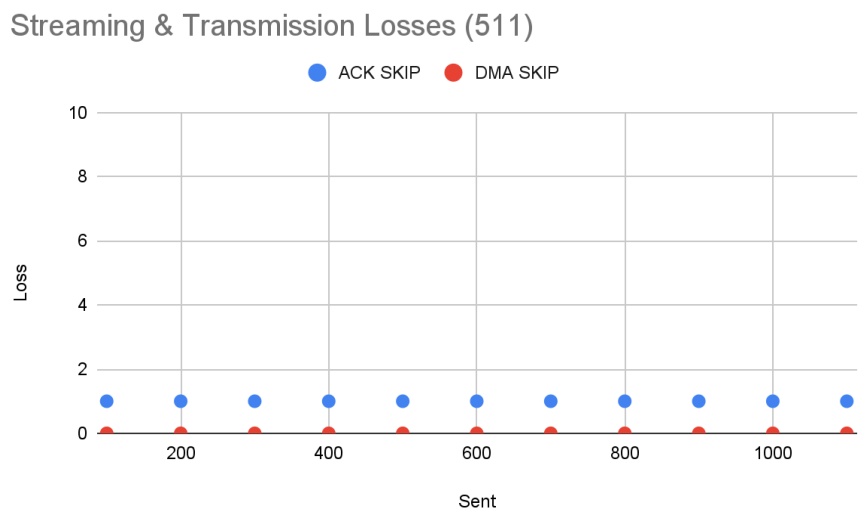

Figure 1. ACK and DMA skip count reports every 100 transmissions for 511-byte messages.

Figure 2. ACK and DMA skip count reports every 100 transmissions for 255-byte messages.

Figure 3. ACK and DMA skip count reports every 100 transmissions for 127-byte messages.

The observed transmission loss rate for 127-byte messages converges toward a constant value, as is evident by the linear relationship between transmission and total loss. The loss rate is approximately 47% measured at 2900 transmissions.

Round-trip time (RTT) was reported for every ACK event for each message length test. The following histograms display the distributions of reported RTT.

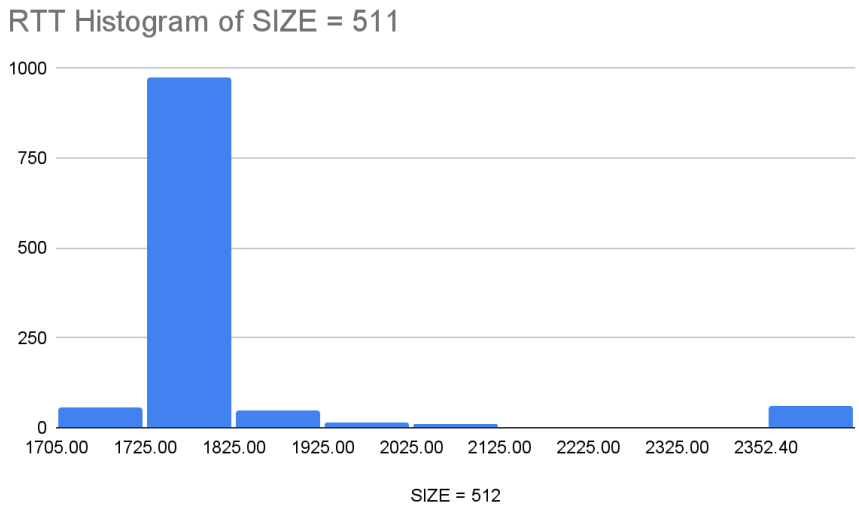

Figure 4. RTT Histogram for 511-byte messages.

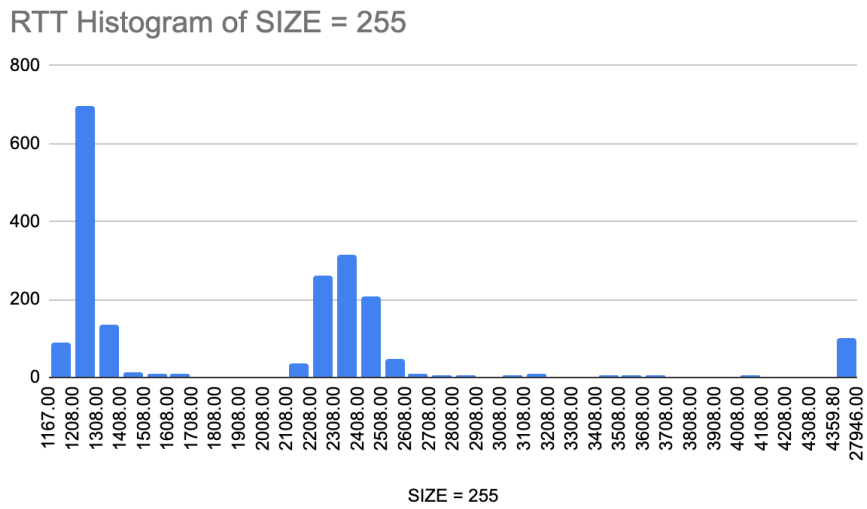

Figure 5. RTT Histogram for 255-byte messages.

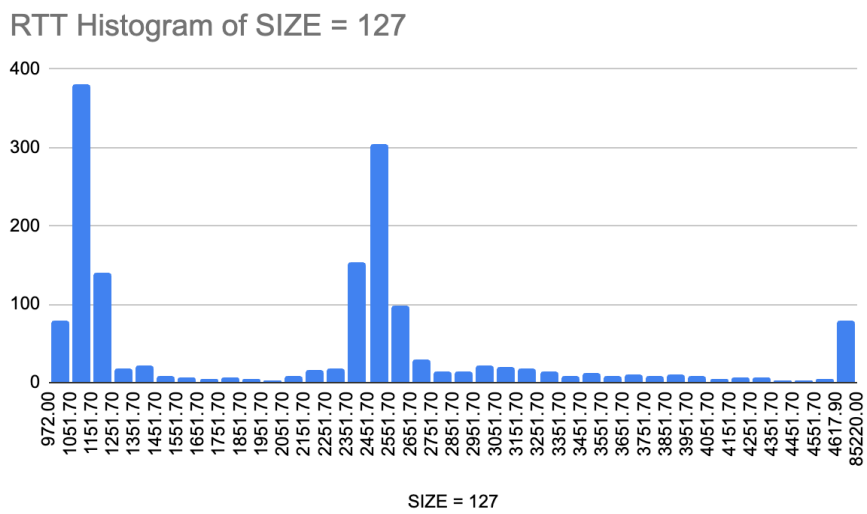

Figure 6. RTT Histogram for 127-byte messages.

The asynchronous nature of the software led to multimodal RTT distributions. The periodicity of these modes suggests occasional misalignment of message timestamps. These are likely largely due to lost or unacknowledged messages. Since the RTT for 511-byte messages is of a single mode, the "correct" RTT ranges for the other trials can be assumed to be in the same ballpark. These are summarized in Table 1.

Table 1.

Message Length (bytes)	127	255	511
Median RTT (ms)	1.05	1.25	1.8

Discussion

The DMA skips being zero suggests that the software at least *called* the UDP send function with enough throughput to match the 96kBps input rate. This suggests that any loss had to do mostly with the receiver.

The high loss rate of the 127-byte messages most likely has to do with the software overhead of allocation and data preparation. Responding to four messages for every DMA buffer fill event was simply impractical. In this system, DMA buffers are filled up every 5.31ms. Even with RTTs averaging 1.05ms, 4.2ms of transmission per batch leaves very little room for other software to run. With the lwip stack and protothread scheduler performing background operations and with serial output all running on the same core, transmission failures are to be expected.

Even though there was a significant tradeoff between RTT and message size, the relationship is not directly proportional. As such, the larger messages proved to be better suited to this application.